

CY Cergy-Paris Université

RAPPORT

pour le projet Génie Logiciel
Licence Informatique, 2 Année

sur le sujet

LA DOMOTIQUE

rédigé par

TRAORE Mariam
AIT TALEB Yassine
SADLI Agnies

Avril 2026

Table des matières

1	Introduction	3
1.1	Contexte du projet	3
1.2	Objectif du projet	3
1.3	Organisation du rapport	3
2	Spécification du projet	4
2.1	Notions de base et contraintes du projet	4
2.1.1	Fonctionnement général du logiciel	4
2.1.2	Notions et terminologies de base	4
2.1.3	Contraintes et limitations connues	6
2.2	Fonctionnalités attendues du projet	6
3	Conception et mise en œuvre	9
3.1	Architecture globale du logiciel	9
3.2	Classes de données	9
3.3	IHM Graphique	10
3.4	Conception des traitements (processus)	11
4	Manuel utilisateur	14
4.1	Lancement de l'application	14
4.2	Fenêtre de simulation	14
4.3	Contrôles disponibles	14
4.4	Lecture des informations affichées	14
5	Déroulement du projet	15
5.1	Réalisation du projet par étapes	15
5.2	Répartition des tâches, communication et intégration	16
6	Répartition des tâches, communication et intégration	16
6.1	Répartition des tâches	16
6.2	Communication	16
6.3	Intégration	16
7	Conclusion et perspectives	18
7.1	Résumé du travail réalisé	18
7.2	Améliorations possibles du projet	18

Table des figures

1	Architecture du logiciel	9
2	Diagramme de classes	10
3	Lanceur du logiciel	11
4	Fenêtre principale	12
5	Capture de l'IHM Graphique du logiciel	15

Liste des tableaux

1	États d'humeur du maître de maison et influence sur le système	5
2	Évolution des besoins physiologiques du maître de maison	5
3	Routine journalière en semaine	5
4	Routine journalière le weekend	6
5	Actions automatiques du système selon l'état du maître	7

Remerciements

Les auteurs de ce projet voudraient remercier chaleureusement **M. LIU** , encadrant de ce projet, pour ses conseils et sa disponibilité tout au long de cette réalisation.

1 Introduction

1.1 Contexte du projet

Ce rapport présente le travail réalisé dans le cadre du projet de Génie Logiciel de deuxième année de Licence Informatique. Le projet porte sur la conception et la mise en œuvre d’une simulation domotique en Java, baptisée *Simulation Domotique*, dans laquelle une maison intelligente prend en charge la vie quotidienne d’un occupant appelé *le maître*.

Le projet a été conçu comme un terrain d’application des concepts vus en cours : programmation orientée objet, patrons de conception, modularisation, tests unitaires et interface graphique avec **Swing**. Il s’agit également d’un exercice complet de génie logiciel, depuis l’écriture du cahier des charges jusqu’à la livraison d’un logiciel fonctionnel et testé.

La simulation se déroule à l’intérieur d’une maison composée de plusieurs pièces (chambre, salle de bain, salon, cuisine) reliées par un couloir. Le maître y évolue de façon autonome : il suit une routine journalière différenciée entre semaine et week-end, ses besoins vitaux décroissent au fil du temps, et des imprévus aléatoires peuvent venir bouleverser son emploi du temps. L’ensemble est piloté par une horloge interne et observable depuis une interface graphique dédiée.

1.2 Objectif du projet

L’objectif principal est de produire un logiciel de simulation à la fois fidèle à un cahier des charges précis et structuré selon de bonnes pratiques de génie logiciel. Concrètement, le projet doit permettre :

- De simuler la journée d’un occupant à l’intérieur d’une maison connectée décomposée en pièces avec des meubles interactifs.
- De faire évoluer en temps réel l’état du maître (énergie, faim, soif, hygiène) et son humeur globale.
- De permettre au système domotique de la maison de proposer des actions au maître selon son état et sa situation.
- De faire suivre au maître une routine différenciée entre les jours de semaine et le week-end, tout en intégrant des imprévus aléatoires.
- D’offrir une interface graphique permettant de visualiser la maison, le déplacement du maître, l’horloge, les statistiques et les notifications de la maison intelligente.
- De garantir la qualité du code par une suite de tests unitaires couvrant chaque composant.

1.3 Organisation du rapport

Le rapport est structuré en six sections principales. La section 2 présente la spécification du projet et les fonctionnalités attendues. La section 3 décrit la conception et la mise en œuvre du logiciel : architecture globale, classes de données, interface graphique et conception des traitements. La section 4 fournit un manuel d’utilisation à destination de l’utilisateur final. La section 5 détaille le déroulement du projet en équipe. Enfin, la section 7 dresse un bilan du travail réalisé et propose des pistes d’amélioration.

2 Spécification du projet

Nous avons présenté l’objectif du projet dans la section 1. Dans cette section, nous présentons la spécification complète de notre logiciel, correspondant au cahier des charges établi en début de projet. Nous y détaillons les notions fondamentales, les contraintes techniques, la modélisation de l’environnement, ainsi que l’ensemble des fonctionnalités attendues du système domotique intelligent.

2.1 Notions de base et contraintes du projet

2.1.1 Fonctionnement général du logiciel

Le logiciel simule une maison intelligente en vue verticale 2D, composée de plusieurs pièces (salon, chambre, cuisine, salle de bain) reliées par un couloir central. Un agent autonome, appelé *Maître de maison*, évolue dans cet environnement selon une routine journalière prédéfinie. Le système domotique analyse en continu l’état du maître — ses besoins physiologiques et son humeur — afin de déclencher automatiquement des actions adaptées sur les équipements connectés de l’habitat.

La simulation se déroule en temps accéléré et configurable : une journée complète de 24 heures simulées correspond à 24 minutes réelles, soit un rapport de 1 minute réelle pour 1 heure simulée. Ce mode de fonctionnement permet d’observer l’ensemble du comportement du système sur une journée complète dans un laps de temps réduit, tout en conservant la cohérence temporelle des événements simulés.

L’architecture logicielle repose sur une séparation claire des responsabilités : un moteur de simulation gère la logique métier (déplacements, besoins physiologiques, déclenchement des actions), une couche graphique assure l’affichage de l’environnement et de l’état du maître, et un module de configuration centralise les paramètres globaux de la simulation. Cette organisation en couches facilite la maintenabilité du code et l’évolution future du projet.

2.1.2 Notions et terminologies de base

Domotique La domotique rassemble l’ensemble des techniques permettant de contrôler, programmer et automatiser une habitation. Elle regroupe les domaines de l’électronique, de l’informatique, de la télécommunication et des automatismes. Le terme est issu du latin *domus* (domicile) et du suffixe *-tique* (technique). Dans le cadre de ce projet, la domotique est simulée logiciellement : les équipements physiques sont remplacés par des objets Java dont l’état est géré et affiché en temps réel.

Maître de maison Il s’agit de l’utilisateur principal du système domotique, pour lequel le système s’adapte et se personnalise. Il possède un profil complet incluant son état d’humeur ainsi que ses besoins physiologiques : faim, soif, fatigue et hygiène. Chacun de ces paramètres est modélisé par une barre de jauge évoluant de 0 à 100%, dont la valeur influence directement les décisions du système. Un seul maître de maison est géré dans cette simulation.

État d’humeur L’humeur représente l’état émotionnel du maître à un instant donné. Elle est définie aléatoirement au début de la simulation, c’est-à-dire au réveil du maître, et évolue dynamiquement en fonction des besoins physiologiques, de l’état de santé et des imprévus survenus au cours de la journée. Un état d’humeur négatif est engendré par une dégradation des paramètres physiologiques, tandis qu’une stabilité de ceux-ci favorise une humeur positive. Les quatre états d’humeur possibles sont présentés dans le tableau 1.

Besoins physiologiques Les besoins du maître de maison sont modélisés sous forme de barres de jauge évoluant continuellement au cours de la simulation. Chacune évolue selon un taux horaire défini, comme présenté dans le tableau 2.

État d'humeur	Description et influence sur le système
Joyeux	État positif. Le système favorise les activités conviviales et les repas colorés. L'ambiance lumineuse est vive et chaleureuse.
Triste	État négatif. Le système propose des activités réconfortantes, des plats doux et une ambiance apaisante.
En colère	État négatif. Le système limite les stimulations extérieures, propose des repas consistants et une musique calme.
Neutre	État stable. Le système propose de la nouveauté et de la variété pour stimuler la curiosité du maître.

TABLE 1 – États d'humeur du maître de maison et influence sur le système

Besoin	Taux de diminution	Condition de remontée
Faim	−7% par heure simulée	Après un repas
Soif	−15% par heure simulée	Après hydratation
Énergie	−5 à −20% par heure selon l'activité	Durant le sommeil
Hygiène	−3% par heure simulée	Après une douche ou un bain

TABLE 2 – Évolution des besoins physiologiques du maître de maison

Objets connectés Plusieurs équipements domotiques sont simulés dans la maison. Chaque objet possède un état actif ou inactif géré automatiquement par le système :

- **Éclairage** : les lumières de chaque pièce peuvent être allumées ou éteintes automatiquement. Leur intensité (forte ou faible) ainsi que leur couleur (LED colorées) sont également gérées en fonction de l'humeur et de l'heure.
- **Chauffage** : la maison régule automatiquement la température intérieure afin de maintenir un confort ambiant, ou de l'adapter à l'humeur du maître.
- **Volets** : leur ouverture et fermeture sont automatisées selon la météo ou l'horaire de la journée.
- **Enceinte connectée** : l'ambiance sonore est gérée selon plusieurs modes : calme, dynamique ou relaxant, selon le contexte émotionnel du maître.

Routine journalière Le maître de maison suit une routine prédéfinie qui varie selon le type de journée. Les tableaux 3 et 4 présentent respectivement le déroulement d'une journée de semaine et d'une journée de weekend.

Horaire	Activité
07 :00	Réveil — ouverture automatique des fenêtres. Barre de fatigue au maximum, barre d'hygiène basse. Le système propose une douche.
07 :05	Douche (20 min) — barre d'hygiène au maximum. Barre de faim basse. Le système propose un petit-déjeuner.
07 :25	Petit-déjeuner — barre de faim au maximum.
07 :40	Départ au travail — la porte de sortie s'ouvre.
18 :40	Retour à domicile — humeur basse. Le système propose une douche, un repas, une boisson et une activité relaxante.
20 :00	Dîner.
22 :00	Coucher — fin de journée.

TABLE 3 – Routine journalière en semaine

Horaire	Activité
10 :00	Réveil tardif — proposition de douche.
10 :30	Petit-déjeuner.
11 :00	Travail à domicile dans le bureau.
14 :00	Déjeuner.
14 :30–15 :30	Sieste — légère hausse de la barre d'énergie.
16 :00–18 :00	Sortie extérieure — diminution des barres de fatigue, d'hygiène et d'humeur.
18 :00	Retour — activité relaxante proposée par le système.
21 :00	Dîner puis coucher à 22 :00.

TABLE 4 – Routine journalière le weekend

Imprévus Des événements non planifiés peuvent survenir au cours de la journée et perturber la routine normale du maître. Le système doit détecter ces situations et réagir de manière adaptée. Les imprévus pris en compte dans la simulation sont les suivants :

- **Réveil en retard** : entraîne le décalage de l'ensemble des activités matinales et une dégradation plus rapide des besoins physiologiques.
- **Message téléphonique urgent** : sollicitation professionnelle imprévue en dehors des horaires habituels, nécessitant une réorganisation des priorités.
- **Retour tardif du travail** : décalage du dîner, annulation ou report des activités du soir.

Chaque imprévu influence négativement l'humeur du maître et modifie dynamiquement les priorités d'action du système.

2.1.3 Contraintes et limitations connues

:

Outils de développement utilisés :

1. Langage de programmation : Java
2. Environnement de développement : Eclipse
3. Outil de rédaction du rapport : LATEX

2.2 Fonctionnalités attendues du projet

Cette sous-section présente l'ensemble des fonctionnalités que le système domotique doit implémenter. Celles-ci sont regroupées en deux catégories : les fonctionnalités générales de simulation, et les actions automatiques déclenchées en fonction de l'état du maître.

Fonctionnalités générales Le système domotique doit assurer les fonctionnalités suivantes :

- Modéliser graphiquement un appartement en 2D composé de plusieurs pièces distinctes, d'un couloir central et d'une sortie.
- Simuler le déplacement autonome du maître de maison dans l'appartement, en respectant les contraintes physiques telles que les murs, les portes et les bordures de la carte.
- Gérer en continu l'évolution des besoins physiologiques du maître selon les taux horaires définis dans le tableau 2.
- Calculer dynamiquement l'état d'humeur du maître à partir de ses paramètres physiologiques et des événements survenus.
- Gérer la routine journalière du maître selon le type de journée (semaine ou weekend), en déclenchant les activités aux horaires prévus.

- Détecter et gérer les imprévus en notifiant le maître et en réorganisant les priorités du système en conséquence.
- Permettre à l'utilisateur de mettre en pause, reprendre ou accélérer la simulation via l'interface graphique.
- Afficher en temps réel l'état du maître (barres de besoins, humeur, heure simulée) ainsi que l'état des équipements connectés.

Actions automatiques selon l'état du maître Le système surveille en permanence l'état du maître et déclenche des actions spécifiques dès qu'un seuil prédéfini est atteint. L'ensemble de ces déclencheurs et de leurs effets est récapitulé dans le tableau 5.

Condition déclenchante	Action du système	Effet
Barre de faim < 40% aux heures de repas	Proposition d'un repas adapté à l'humeur	Remplissement de la barre de faim, légère hausse de l'humeur
Barre de soif < 50%	Proposition d'une boisson adaptée à l'humeur	Remplissement de la barre de soif
Barre d'énergie < 20% ou heure > 22h30	Activation du mode nuit (lumières tamisées, volets fermés, musique douce)	Remplissement de la barre de sommeil
Niveau d'hygiène < 20%	Proposition d'un bain immédiat	Hausse du niveau d'hygiène et effet positif sur l'humeur
Humeur négative (tristesse ou colère)	Ajustement de l'ambiance intérieure (éclairage, température, musique)	Retour progressif à un état émotionnel stable
Survenue d'un imprévu	Notification du maître et réorganisation des priorités	Limitation de l'impact sur le confort et le bien-être

TABLE 5 – Actions automatiques du système selon l'état du maître

Comme illustré dans le tableau 5, les actions du système sont hiérarchisées selon un ordre de priorité strict. Les besoins vitaux faim, soif et santé sont toujours traités en premier, suivis du sommeil, de l'hygiène, du bien-être émotionnel, et enfin des activités de loisir. Cette organisation garantit une simulation cohérente, réaliste et centrée sur le bien-être du maître de maison.

Adaptation des repas selon l'humeur Le système ne se contente pas de proposer un repas lorsque la barre de faim est basse : il adapte également le type de repas à l'état émotionnel du maître. Ainsi :

- **Joyeux** : repas coloré et convivial (pizza, tacos, burgers).
- **Triste** : plat réconfortant à texture douce (purée, lasagnes, hachis parmentier), avec éventuellement un dessert sucré si l'humeur est particulièrement basse.
- **En colère** : plat consistant et épicé (kebab, steak, wrap).
- **Neutre** : cuisine du monde pour stimuler la curiosité (mexicain, libanais, asiatique).

Adaptation des boissons selon l'humeur De la même manière, le choix de la boisson proposée est conditionné par l'état émotionnel du maître :

- **Humeur stimulante (joyeux, en colère)** : boisson rafraîchissante et pétillante (soda, eau aromatisée, smoothie aux fruits).
- **Humeur basse (triste, neutre)** : boisson réconfortante et énergisante (thé vert, thé noir, smoothie exotique).

L'ensemble de ces fonctionnalités constitue le cœur du système domotique intelligent et adaptatif développé dans ce projet. Leur implémentation repose sur une architecture orientée objet rigoureuse, détaillée dans la section suivante.

3 Conception et mise en œuvre

3.1 Architecture globale du logiciel

Dans la figure 1, on peut voir un programmeur très occupé par son travail.

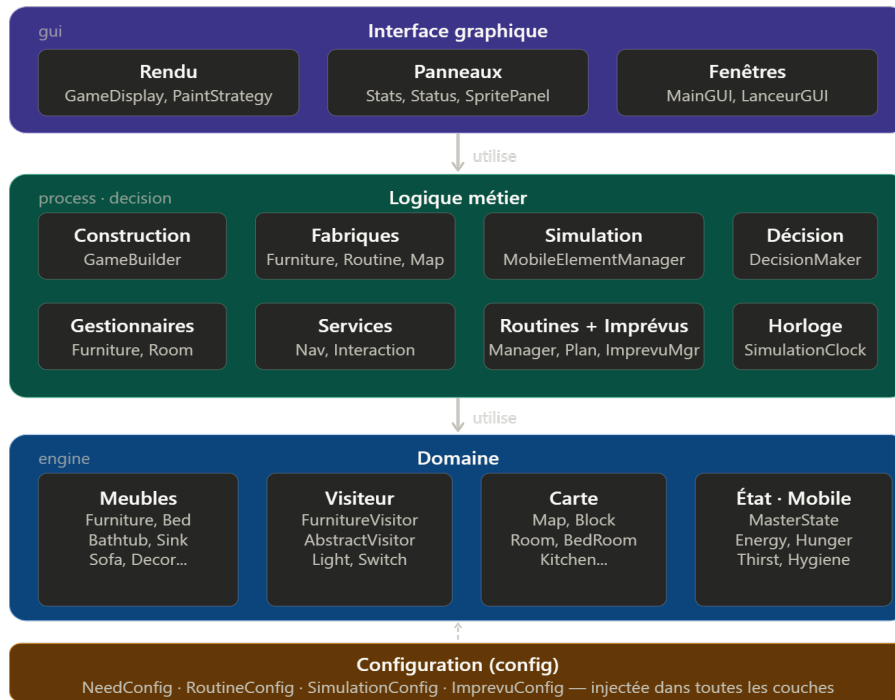


FIGURE 1 – Architecture du logiciel

3.2 Classes de données

Conception et mise en œuvre des classes de données.

Les classes de données du projet modélisent les entités manipulées par la simulation. Elles sont volontairement simples et regroupées dans le paquetage **engine**.

Carte et blocs La classe **Map** représente la grille de la maison et expose des méthodes utilitaires pour tester si un bloc est sur un bord. Chaque **Block** stocke ses coordonnées (ligne, colonne) et constitue l'unité de base de tous les déplacements.

Pièces La classe abstraite **Room** décrit une pièce rectangulaire (ligne / colonne de début et de fin), une porte et une lumière. Quatre classes filles concrétisent ce modèle : **BedRoom**, **BathRoom**, **LivingRoom** et **Kitchen**. La création passe par une fabrique **RoomFactory**, ce qui isole la logique d'instanciation.

Meubles La classe abstraite **Furniture** factorise un nom, une liste de blocs occupés et un état actif / en veille. Six meubles concrets en héritent : **Bed**, **Bathtub**, **Oven**, **Sofa**, **Television** et **WaterBottle**. À ces meubles s'ajoutent un objet **Light** pour la lumière des pièces et un **Switch** qui permet d'agir manuellement sur la lumière du couloir.

Élément mobile Le maître est représenté par la classe **Master** qui hérite de la classe abstraite **MobileElement**. Cette dernière n'expose qu'une position courante (un **Block**), ce qui prépare une extension future à plusieurs occupants.

État du maître Le paquetage `engine.state` contient une classe par besoin (`Energy`, `Hunger`, `Thirst`, `Hygiene`) et la classe `MasterState` qui les agrège. Chaque besoin maintient une valeur entre 0 et 100, qui décroît à intervalles réguliers et qui peut atteindre un seuil dit critique. `MasterState` expose en plus le calcul de l'humeur globale et la détection du besoin le plus critique.

Horloge et événements La classe `SimulationClock` maintient l'heure courante (heure, minute, indice du jour) et est avancée d'une minute à chaque tour. Le paquetage `engine.event` définit la classe `Imprevu` (un imprévu = code, nom, description) et le `ImprevuManager` qui maintient le catalogue des imprévus disponibles et applique leurs effets.

Le schéma 2 représente un diagramme de classe liant les classes de données entre elles.

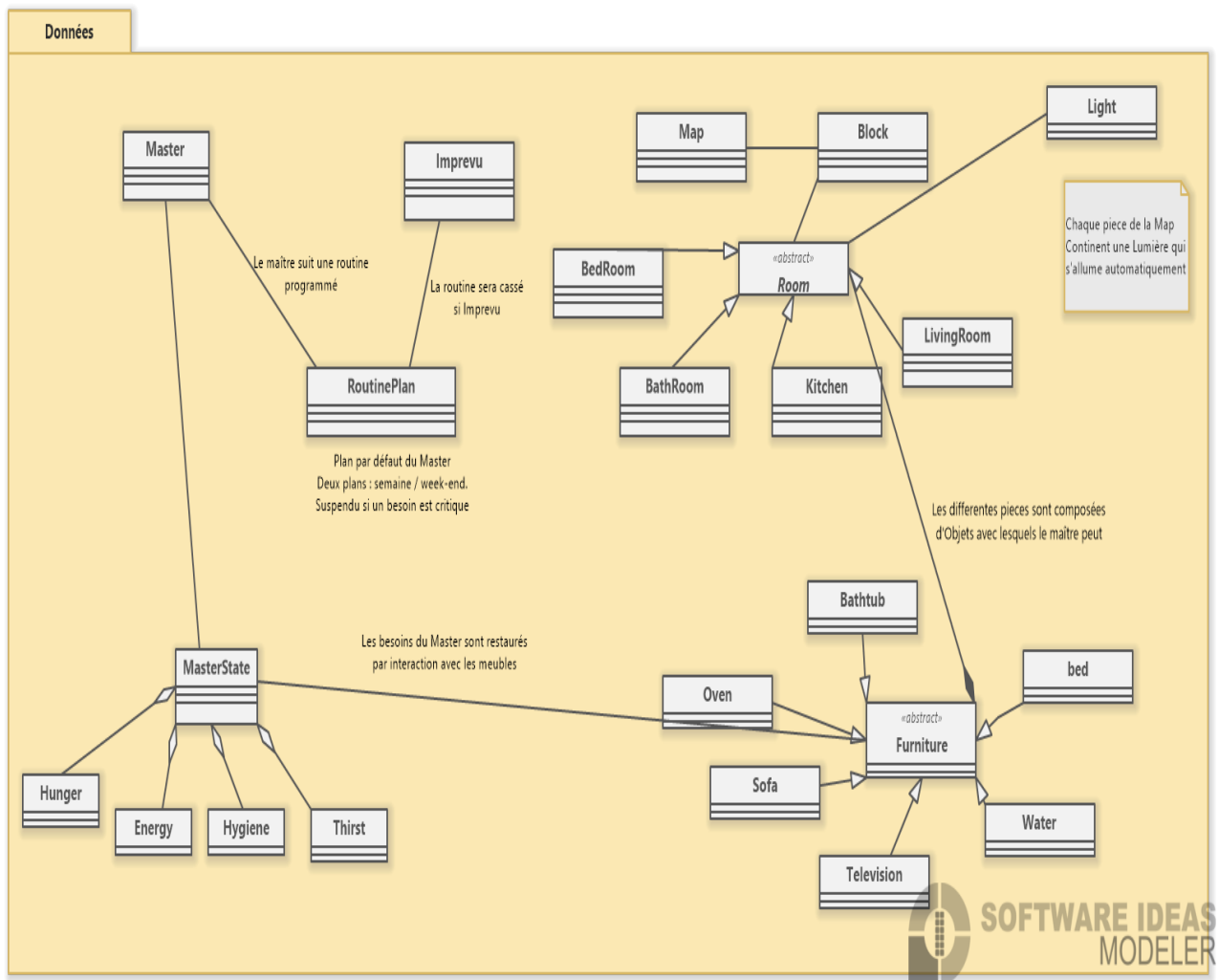


FIGURE 2 – Diagramme de classes

3.3 IHM Graphique

Conception et mise en œuvre de l'IHM graphique de votre projet.

L'interface est construite avec `Swing` et organisée en deux fenêtres principales : un lanceur `LauncherGUI` et la fenêtre de simulation `MainGUI`. Le découpage en sous-panneaux suit la logique du tableau de bord : à gauche se trouve le rendu de la maison, à droite un bandeau latéral d'informations, et en bas une zone de notifications et de routine.

Lanceur La fenêtre d'accueil propose trois boutons : *Play* pour lancer la simulation, *Crédits* pour ouvrir une fenêtre listant les auteurs (gérée par `CreditClass`), et *Quitter* pour fermer l'application. Chaque bouton est associé à un `ActionListener` dédié.

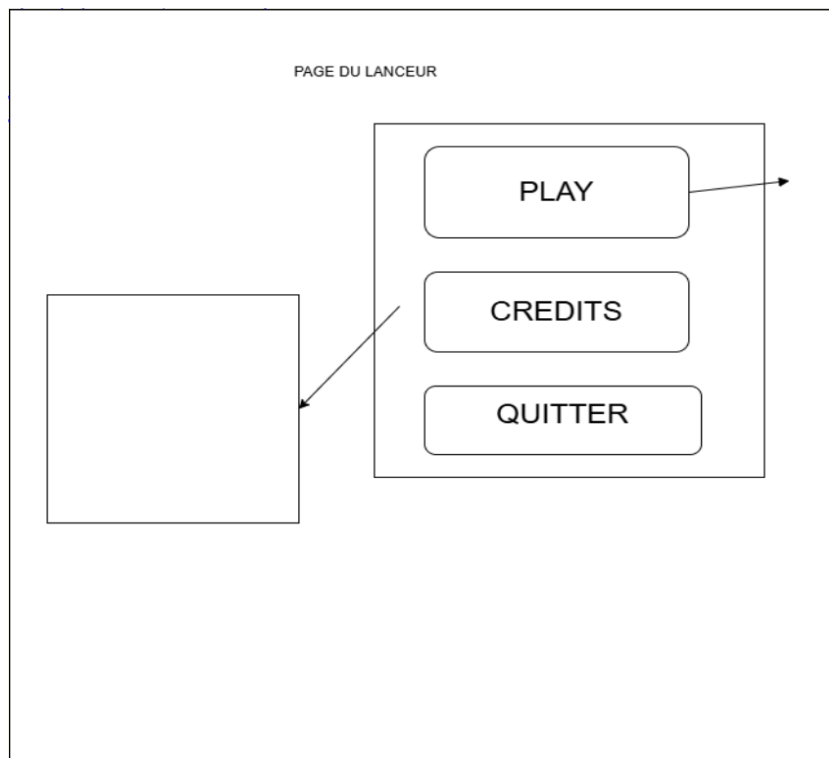


FIGURE 3 – Lanceur du logiciel

Fenêtre principale La fenêtre `MainGUI` se décompose en plusieurs panneaux :

- `GameDisplay` : panneau central qui dessine la carte, les pièces, les meubles et le maître à l'aide d'un `PaintStrategy` et d'un `SpritePanel` pour les sprites.
- `StatsPanel` : barres d'état des quatre besoins et indicateur d'humeur.
- `FurnitureStatusPanel` : état courant des meubles (en veille / actif, plat ou émission en cours).
- Cartes latérales : horloge, indicateur de vitesse, contrôles (Start / Pause, Clear, Speed Up).
- Onglets en bas : *Routine* (étape courante, prochaine étape, type de jour, imprévu) et *Consignes* (notifications horodatées de la maison).

Boucle de rendu La méthode `run()` de `MainGUI` maintient deux pas de temps distincts : un pas « logique » rythmé par `GAME_SPEED` et un pas « rendu » rythmé par `FRAME_DURATION`. À chaque pas logique, `manager.nextRound()` est appelé ; à chaque pas rendu, `refreshDisplay()` met à jour l'horloge, la zone de routine, les notifications et redessine les panneaux.

3.4 Conception des traitements (processus)

La logique du jeu est portée principalement par `MobileElementManager`, qui orchestre à chaque tour la mise à jour de l'horloge, la mise à jour des besoins, l'évaluation de l'imprévu du jour, la mise à jour des lumières, puis la prise de décision et l'action correspondante.

Calcul de l'humeur L'humeur globale du maître est calculée comme une moyenne pondérée des quatre besoins, après application d'une pénalité pour les valeurs en dessous de 30 (la valeur effective est alors multipliée par 0,4).

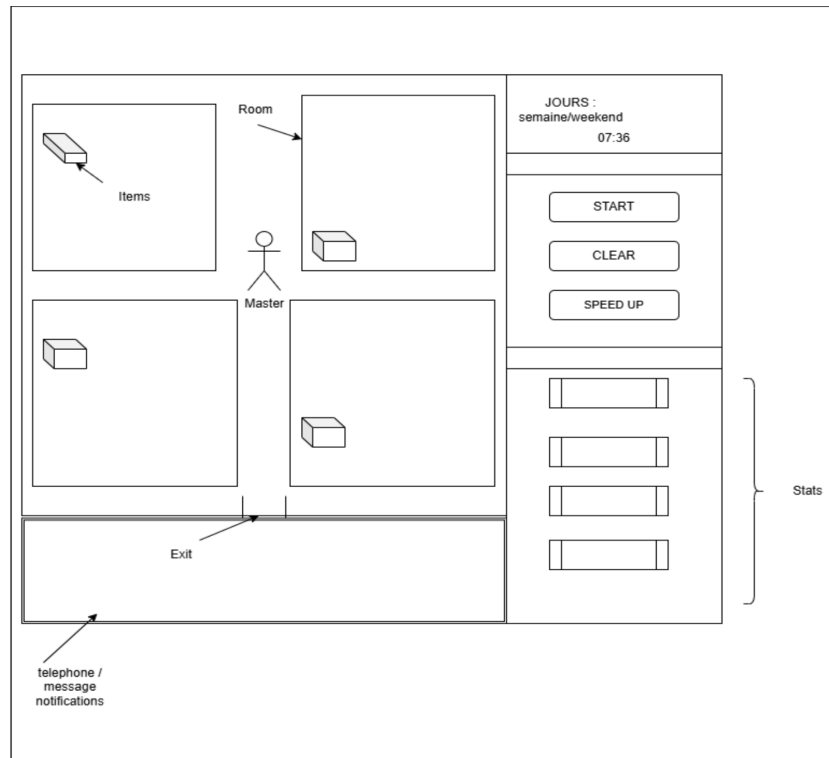


FIGURE 4 – Fenêtre principale

Dans la formule mathématique 1, on peut voir le calcul de l'humeur, où E , F , S et H désignent respectivement les valeurs (éventuellement pénalisées) d'énergie, faim, soif et hygiène.

$$Mood = 0,28 \cdot E + 0,25 \cdot F + 0,30 \cdot S + 0,17 \cdot H \quad (1)$$

Dans 1, on décrit un exemple d'algorithme : la boucle principale exécutée à chaque tour de simulation par `MobileElementManager`.

Patrons de conception utilisés Le projet exploite plusieurs patrons de conception classiques :

- *Builder* pour l'assemblage initial du jeu (`GameBuilder`).
- *Factory* pour la création des pièces et des meubles (`RoomFactory`, `FurnitureFactory`).
- *Strategy* pour les stratégies de décision (`DecisionStrategy` et ses trois implémentations).
- *Visitor* pour les opérations sur les meubles (`FurnitureVisitor` avec `TickVisitor`, `ResetVisitor`, `EffectVisitor`, `IsDoneVisitor`).
- *Service* pour découpler les responsabilités lourdes du `MobileElementManager` (`MasterNavigationService`, `FurnitureInteractionService`).

Algorithm 1 Tour de simulation : *nextRound*

Require: Une horloge C , un état S , un gestionnaire d'imprévu I , une routine R , un *DecisionMaker* D et un maître M

Ensure: Le maître a effectué une action pour ce tour

```
 $C.nextRound()$ 
 $S.updateStates()$ 
 $I.evaluateDailyImprevu(C.getDayIndex())$ 
 $updateLight()$ 
if  $M$  est en interaction then
     $continueInteraction()$ 
    return
end if
 $step \leftarrow R.getCurrentStep()$ 
if  $step$  est obligatoire et prête then
     $handleMandatoryStep(step)$ 
    return
end if
if  $M$  est à la maison then
     $decision \leftarrow D.decideNextAction()$ 
    if  $decision$  a une cible then
        if  $M$  est près de la cible then
             $startInteraction(decision.target)$ 
        else
             $moveCloser(nextWaypoint)$ 
        end if
    end if
end if
end if
```

4 Manuel utilisateur

Dans la figure 5 ci dessous, on peut voir l'ihm graphique et tout ses composantes qu'on décrira ci bas :

4.1 Lancement de l'application

L'application se lance en exécutant la classe `TestGame` (paquetage `test`). Cette classe ouvre la fenêtre d'accueil intitulée « Simulation Domotique ». Trois boutons sont alors proposés :

- *Play* : démarre la simulation et ouvre la fenêtre principale.
- *Crédits* : ouvre une fenêtre listant les auteurs du projet.
- *Quitter* : ferme l'application.

4.2 Fenêtre de simulation

Une fois la simulation lancée, la fenêtre principale est composée :

- D'une zone centrale qui affiche la maison vue de dessus, avec ses pièces, ses meubles, ses lumières et le maître qui s'y déplace.
- D'un bandeau latéral droit qui regroupe l'horloge, l'indicateur de vitesse, la barre d'état des besoins, l'état des meubles et les boutons de contrôle.
- D'un bandeau inférieur à onglets : l'onglet *Routine* indique le type de jour, l'imprévu éventuel, l'étape courante et la prochaine étape ; l'onglet *Consignes* affiche les notifications horodatées de la maison.

4.3 Contrôles disponibles

Trois boutons sont mis à disposition de l'utilisateur dans la carte *Contrôles* :

- *Start / Pause* : démarre la simulation ou la met en pause. Au premier clic, un fil de simulation est lancé. Un nouveau clic met le fil en pause.
- *Clear* : remet la simulation dans son état initial (heure, position du maître, besoins, routine), sans fermer la fenêtre.
- *Speed Up* : multiplie la vitesse de simulation par un facteur (1, 2 ou 3) et boucle. La carte *Actual Speed* affiche en permanence le multiplicateur courant.

4.4 Lecture des informations affichées

Horloge La carte *Heure* affiche le temps interne de la simulation au format HH:MM. La simulation démarre par défaut à 07 :30 du jour 0.

Barre d'état Quatre jauges représentent les besoins du maître (énergie, faim, soif, hygiène). Chaque valeur diminue régulièrement et passe en zone critique en dessous de 20. L'humeur globale est également affichée sous forme d'étiquette (*Very unhappy*, *Uneasy*, *Neutral*, *Happy*, *Excellent*).

État des meubles La carte *État des objets* liste les meubles de la maison avec leur état courant (en veille / actif). Pour le four, le plat en cours de cuisson est précisé ; pour la télévision, l'émission diffusée.

Routine et notifications L'onglet *Routine* permet de suivre la progression du maître dans sa journée. L'onglet *Consignes* accumule les messages textuels que la maison adresse à son occupant (par exemple : « Le maître doit dormir », « Imprévu du jour : Coupure de courant »).

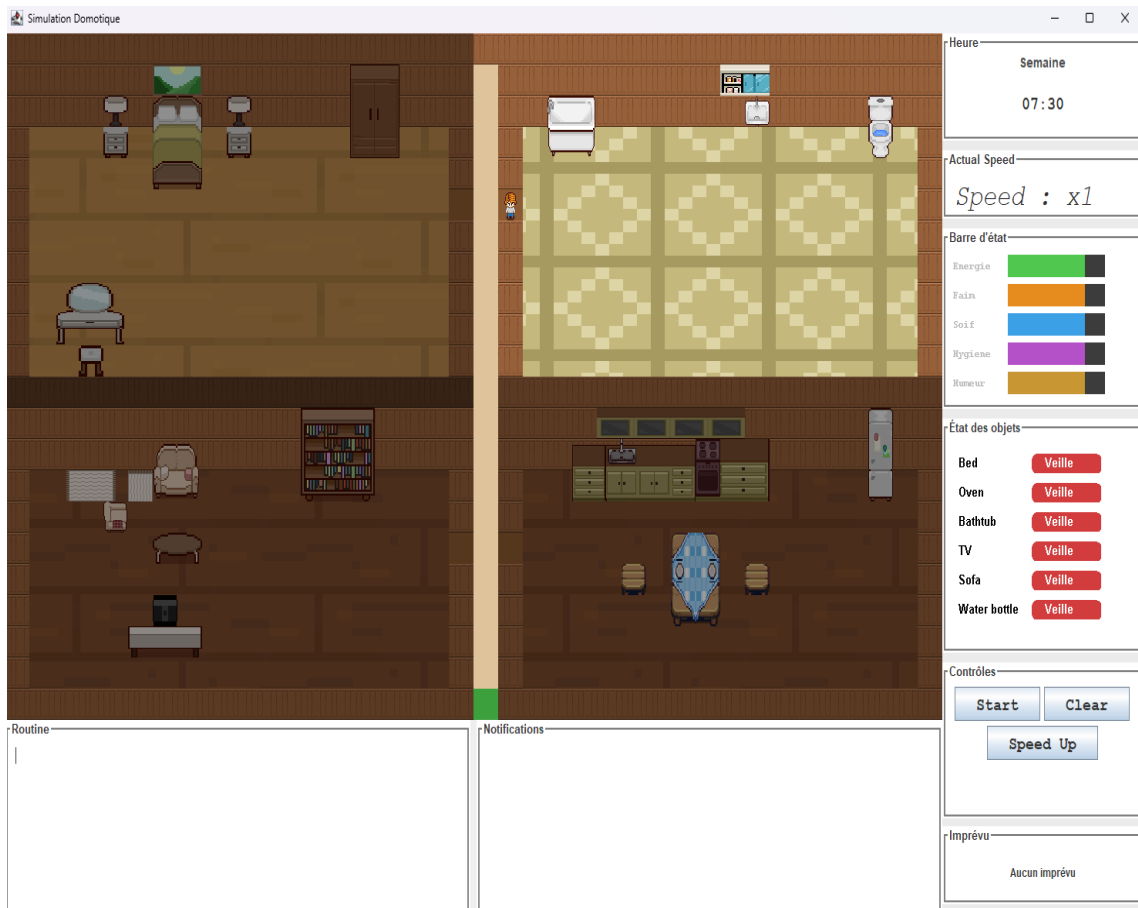


FIGURE 5 – Capture de l’IHM Graphique du logiciel

5 Déroulement du projet

Dans cette section, nous décrivons comment le projet a été réalisé en équipe : la répartition des tâches, la synchronisation du travail en membres de l’équipe, etc.

5.1 Réalisation du projet par étapes

Le projet a été conduit selon un découpage en étapes successives, en s’inspirant d’un cycle de développement incrémental.

Étape 1 — Analyse et cahier des charges Étude du sujet, rédaction du cahier des charges, identification des grandes entités (maison, pièces, meubles, maître, besoins, routine) et premières maquettes papier de l’interface.

Étape 2 — Squelette du moteur Mise en place du packaging `config`, de la classe `Map`, des classes `Room` et de leurs filles, ainsi que des classes de meubles. Création du `GameBuilder` pour assembler la carte et les meubles.

Étape 3 — État du maître et horloge Implémentation des quatre besoins (`Energy`, `Hunger`, `Thirst`, `Hygiene`), de `MasterState` (humeur, besoin critique) et de `SimulationClock`.

Étape 4 — Routine et imprévus Conception des classes `RoutineStep`, `RoutinePlan` et `RoutineManager`, avec deux modèles (semaine et week-end). Ajout du `ImprévuManager` et des cinq imprévus (*Grasse matinée*, *Visite surprise*, *Coupure de courant*, *Journée malade*, *Journée pressée*).

Étape 5 — Décision et navigation Mise en place du pipeline de décision (`DecisionMaker` avec trois `DecisionStrategy`) et du `MasterNavigationService`.

Étape 6 — Sélecteurs d’humeur et services Ajout des sélecteurs (`MoodFoodSelector`, `MoodShowSelector`) et du `FurnitureInteractionService` basé sur le patron *Visitor*.

Étape 7 — Interface graphique Conception du `LauncherGUI`, du `MainGUI` et de tous les panneaux (`GameDisplay`, `StatsPanel`, `FurnitureStatusPanel`, etc.).

Étape 8 — Tests et qualité Rédaction de la suite de tests `DomotiqueTestSuite`, qui regroupe une vingtaine de classes de tests `JUnit` couvrant les besoins, l’horloge, la routine, les imprévus, les sélecteurs, les meubles, la navigation et l’intégration via le `MobileElementManager`.

Étape 9 — Rédaction du rapport Mise en forme du rapport en $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$, intégration des schémas et relectures.

5.2 Répartition des tâches, communication et intégration

6 Répartition des tâches, communication et intégration

6.1 Répartition des tâches

Le projet a été divisé en plusieurs grands domaines fonctionnels, chacun confié à un ou plusieurs membres de l’équipe selon leurs affinités et compétences.

- **Gestion des besoins et système de décision** : développement des états du personnage (énergie, faim, soif, hygiène), mise en place du système de routine et implémentation du mécanisme de priorisation entre besoins critiques et routine.
- **Interface graphique et rendu visuel** : conception de la fenêtre principale, réalisation des panneaux d’affichage (statistiques, état des meubles, notifications), intégration des sprites et des animations du personnage.
- **Navigation et interactions** : gestion des pièces et des meubles, implémentation des déplacements du maître et de la logique d’interaction avec les objets de la maison.

Chaque membre a pu contribuer à l’ensemble du projet tout en étant référent sur son domaine principal. Cette répartition a permis une progression parallèle et efficace.

6.2 Communication

La communication au sein du groupe a été régulière et fluide. Plusieurs outils ont été utilisés pour faciliter les échanges et le suivi du projet :

- Un serveur `Discord` dédié pour les discussions quotidiennes, le partage rapide de documentation et la résolution de problèmes en temps réel.
- Des réunions hebdomadaires en présentiel pour faire le point sur l’avancement, valider les choix d’architecture et répartir les tâches à venir.
- Un dépôt `Git` centralisé pour versionner le code et suivre l’historique des modifications.

Cette organisation a maintenu une bonne coordination et a permis de respecter les délais impartis.

6.3 Intégration

L’intégration des différents modules a été réalisée de manière progressive selon une approche itérative :

1. Développement individuel sur des branches dédiées.
2. Tests unitaires et validation locale des fonctionnalités.
3. Fusion régulière sur la branche principale lors des sessions de travail collectif.

Cette méthode a facilité la détection précoce des incohérences et a garanti la cohérence globale de l'application. L'ensemble des composants a ainsi pu être intégré avec succès pour livrer une simulation fonctionnelle répondant aux exigences initiales.

7 Conclusion et perspectives

7.1 Résumé du travail réalisé

Le projet *La Domotique* a abouti à une simulation domotique fonctionnelle écrite en Java, qui répond à l'ensemble du cahier des charges. La maison se compose de quatre pièces et d'un couloir, et abrite six meubles interactifs avec lesquels le maître interagit selon une logique combinant routine prédéfinie, besoins individuels et imprévus aléatoires.

Sur le plan logiciel, l'application est structurée en paquetages clairement séparés. Le moteur (**engine**) ne dépend ni de l'interface graphique ni de la couche logique, la logique métier (**logic**) repose sur des patrons de conception classiques (Builder, Factory, Strategy, Visitor), et l'interface graphique (**ui**) consomme uniquement une interface (**MobileInterface**) sans connaître les détails internes du moteur. Cette séparation a permis d'écrire une suite de tests unitaires conséquente (la classe **DomotiqueTestSuite** regroupe une vingtaine de classes de tests) qui valide la plupart des composants critiques.

Au-delà du fonctionnement nominal, le projet intègre des éléments d'agrément : choix d'un plat ou d'une émission selon l'humeur, allumage automatique des lumières en fonction de la pièce occupée, notifications horodatées, et indicateurs visuels en temps réel pour l'utilisateur.

7.2 Améliorations possibles du projet

Plusieurs pistes d'amélioration restent ouvertes pour un futur travail.

Navigation plus réaliste Le déplacement actuel utilise une heuristique simple par décomposition colonne / ligne et points de passage par les portes. Une implémentation d'un véritable algorithme de plus court chemin permettrait de gérer des plans de maison plus irréguliers ou d'introduire des obstacles supplémentaires.

Plusieurs occupants L'architecture (classe abstraite **MobileElement**) prépare déjà l'introduction de plusieurs occupants. Il resterait à étendre le **MobileElementManager** pour gérer un ensemble de maîtres et à arbitrer les conflits d'accès aux meubles.

Catalogue d'imprévus extensible Le catalogue d'imprévus est aujourd'hui codé en dur dans **ImprevuManager**. Le rendre configurable (par exemple via un fichier de description) permettrait à l'utilisateur d'ajouter ses propres imprévus sans recompiler le projet.

Persistance La simulation repart actuellement à zéro à chaque lancement. La possibilité de sauvegarder et de recharger un état complet (heure, position, besoins, étape de routine) ouvrirait la voie à des sessions plus longues et à des scénarios pédagogiques.

Améliorations de l'IHM L'interface pourrait s'enrichir d'animations plus poussées sur les sprites, d'un mode plein écran et d'une internationalisation complète des textes affichés (actuellement partagés entre français et anglais selon les modules).

Références

- [Liu25a] Tanxiao Liu. Aircraft. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.
- [Liu25b] Tanxiao Liu. Chrono. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.
- [Liu25c] Tanxiao Liu. Page personnelle. <https://depinfo.u-cergy.fr/~tliu/>, 2025. Consulté en avril 2025.
- [Liu25d] Tanxiao Liu. Train. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.
- [Liu25e] Tanxiao Liu. Tree_v1. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.
- [Liu25f] Tanxiao Liu. Tree_v2. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.
- [Liu25g] Tanxiao Liu. Tree_v3. Logiciel distribué dans le cadre du cours, 2025. Disponible sur la page personnelle de l’auteur.